
Task 5 – Capstone Project & Incident Response

❖ Objective:

Apply all skills in a self-chosen capstone project and simulate an incident response.

❖ Observation:

1. Capstone Project Selection

The first decision in the capstone is which simulated engagement you will build the rest of the project around. Your flow chart offers two tracks. Both are valid, both end in the same Incident Response Simulation phase.

1.1 Vulnerability Assessment of a Test Network

1.1.1 What a Vulnerability Assessment Is

A Vulnerability Assessment (VA) is a systematic review of a system's security weaknesses. It answers the question "what could go wrong here?" by identifying, classifying, and prioritizing vulnerabilities - misconfigurations, missing patches, weak credentials, insecure services - without necessarily proving they are exploitable end-to-end. This distinguishes it from adjacent but different engagement types:

Engagement Type	Primary Question	Depth	Typical Output
Vulnerability Assessment	What weaknesses exist?	Breadth-first, mostly automated scanning	Prioritized list of vulnerabilities (CVSS-scored)
Penetration Test	Can an attacker actually exploit these weaknesses to achieve an objective?	Depth-first, manual exploitation of a subset	Proof-of-concept exploitation chain + business impact
Red Team Exercise	Can the organization detect and respond to a realistic, stealthy adversary?	Full attack lifecycle, evasive, goal-oriented	Detection/response gap analysis
Security Audit	Does the environment meet a specific standard/policy?	Compliance checklist driven	Compliance gap report

For a capstone, a Vulnerability Assessment is usually the right scope: it is safer, faster to complete, and still requires you to demonstrate reconnaissance, scanning, evidence-based reporting, and mitigation reasoning - the same skills a penetration test would require, at a controllable depth.

1.1.2 Types of Vulnerabilities You Will Encounter

- Network-layer: open/unnecessary ports, outdated protocols (SMBv1, Telnet, FTP), weak TLS configurations.
- Host-layer: missing OS patches, weak local account policies, unquoted service paths, insecure permissions.
- Application-layer: injection flaws (SQLi, command injection), XSS, broken authentication, security misconfiguration (OWASP Top 10).
- Configuration/Design: flat networks with no segmentation, default credentials, excessive privileges (least-privilege violations).
- Human-layer: susceptibility to social engineering - this is exactly what Option B tests directly.

1.1.3 Governing Frameworks and Standards

- **PTES (Penetration Testing Execution Standard)**: defines seven phases - pre-engagement, intelligence gathering, threat modelling, vulnerability analysis, exploitation, post-exploitation, reporting - that map directly onto your flow chart's Planning → Implementation → Reporting structure.
- **NIST SP 800-115**: "Technical Guide to Information Security Testing and Assessment" - the U.S. government reference for planning and executing technical security assessments; useful for structuring your Methodology section.
- **OSSTMM**: the Open-Source Security Testing Methodology Manual - emphasizes measurable, repeatable testing ("attack surface metrics").
- **OWASP Testing Guide**: the standard reference specifically for web application vulnerabilities, mapped to the OWASP Top 10.
- **CVE / CWE / CVSS**: CVE (Common Vulnerabilities and Exposures) identifies specific known flaws; CWE (Common Weakness Enumeration) classifies the underlying weakness type; CVSS (Common Vulnerability Scoring System, v3.1/v4.0) converts severity into a 0-10 numeric score.

1.1.4 Tooling by Phase

Phase	Representative Tools	Purpose
Reconnaissance	Nmap (ping/host discovery), Shodan, theHarvester, whois	Map what exists before touching it
Scanning	Nmap (service/version), Nessus, OpenVAS, Nikto (web)	Enumerate services and known-vulnerability matches
Exploitation (controlled)	Metasploit Framework, manual PoC scripts, Burp Suite	Confirm exploitability in a safe, reversible way (lab snapshot first)
Evidence & Reporting	CVSS calculator, screenshots, Dradis/Faraday, Word/Markdown	Turn raw output into defensible findings

1.2 Security Awareness Phishing Simulation

1.2.1 What Phishing Simulation Is

A phishing simulation is a controlled social-engineering exercise: your organization (or, in a lab, a fictitious one) sends a fabricated but realistic phishing email to a defined group of users, then measures who opens it, clicks a link, or submits credentials. Its purpose is not to punish users but to (a) measure human-layer risk quantitatively and (b) drive targeted training. It directly tests the vulnerability category that automated scanners cannot: people.

1.2.2 Social Engineering Theory

Phishing exploits well-documented psychological levers rather than technical flaws:

- Authority - impersonating IT, HR, or an executive.
- Urgency / scarcity - "your account will be locked in 24 hours."
- Fear - fake security alerts, fake invoices/legal threats.
- Trust / familiarity - spoofed sender names, cloned internal branding.
- Curiosity - "see who viewed your profile," fake shared documents.

These map onto Robert Cialdini's principles of influence (authority, scarcity, social proof, reciprocity, commitment, liking) - worth citing directly in an awareness-training report.

1.2.3 Varieties of Phishing (for scenario design)

Type	Channel	Notes
Phishing	Mass email	Broad, low-personalization
Spear phishing	Targeted email	Personalized to a role/individual - higher success rate
Whaling	Email to executives	Targets C-level for financial fraud (BEC)
Vishing	Voice call	Often paired with pretexting
Smishing	SMS	Fake delivery/bank alerts
Pretexting	Any channel	Fabricated scenario building trust before the ask

1.2.4 Tooling and Metrics

- **Tools:** GoPhish (open-source campaign manager), King Phisher, or the Social-Engineering Toolkit (SET) for template/credential-harvesting pages in a lab.
- **Core metrics:** Open rate, click-through rate, credential-submission rate, and - the metric that matters most for a mature security culture - the reporting rate
- **Ethics/legal:** Requires documented management authorization, must avoid harvesting real credentials beyond what's needed for the metric, must not use content that is discriminatory or that could cause genuine distress (e.g., fake termination notices), and should always end with immediate, blame-free training for anyone who clicked.

1.3 Choosing Between the Two Options

Criterion	Vulnerability Assessment	Phishing Simulation
Best if you enjoy...	Networking, systems, exploitation tooling	Psychology, communications, security-awareness design
Lab requirements	VMs, isolated network, vulnerable targets (e.g., Metasploitable)	Mail server/GoPhish instance, mock user list
Primary skill demonstrated	Technical vulnerability discovery & CVSS reasoning	Human-risk measurement & training program design
Feeds into Incident Response as...	A detected exploitation attempt (IDS/EDR alert)	A detected malicious-email / credential-misuse alert

2. Project Planning

Planning converts your chosen scenario into an executable, boundaried, and measurable project. A capstone grader (or a real client) evaluates planning quality as much as technical results - a well-scoped project that finds three real issues outranks an unscoped one that "scanned everything."

2.1 Defining Objectives

Write objectives using the SMART model (Specific, Measurable, Achievable, Relevant, Time-bound). Example for Option A:

Sample Objective Statement

"Identify and document at least five exploitable or high-severity vulnerabilities across three hosts in the isolated capstone lab within a two-week window, and validate detection of at least one exploitation attempt in the SIEM."

2.2 Defining Scope

- In-scope assets: explicit IP ranges / hostnames / URLs / user groups.
- Out-of-scope: production systems, third-party infrastructure, denial-of-service testing (unless explicitly agreed).
- Testing window: dates/times, especially important outside a lab.
- Rules of Engagement (RoE): permitted techniques, escalation contact, "stop" conditions.
- For phishing: exact email templates, sending domain, what happens to captured data, and an opt-out/escalation contact for anyone alarmed.

2.3 Tool Selection

Select tools per phase (reuse the tables in Section 1) and record versions - this becomes an appendix in your final report and makes your results reproducible.

2.4 Timeline

A simple Gantt-style plan for a typical 3-4 week capstone:

Week	Phase	Key Activities
1	Planning	Objectives, scope, RoE, lab build-out, tool installation
2	Implementation - Recon & Scanning	Passive/active recon, port & vulnerability scanning
3	Implementation - Exploitation & Documentation	Controlled exploitation, evidence capture, CVSS scoring
4	Incident Response Simulation & Reporting	Detect/contain/eradicate simulation, write final report

2.5 Diagramming: Network Diagrams vs. ER Diagrams

- **Network / topology diagram:** shows physical or logical infrastructure — hosts, IP addressing, segmentation (VLANs), and where the attacker and monitoring sit relative to targets. Used to define and communicate scope (see Figure 1 in the companion report).
- **Entity-Relationship (ER) diagram:** models the data you will collect and how it relates - e.g., an Asset has many Vulnerabilities, a Vulnerability produces a Finding, a Finding leads to a Mitigation; a Campaign has many Target Users who generate Email Events, which may escalate into an Incident (see Figure 2 in the companion report). Building this before implementation forces you to decide exactly what evidence fields you will capture.

2.6 Risk Assessment and Communication Plan

Identify risks to the plan itself (not just to the target): lab instability, scope creep, accidental impact to a shared network, time overrun. Define who is notified if something unexpected happens, and how findings will be communicated.

3. Implementation

Implementation is where planning becomes evidence. It follows a consistent attacker-emulation lifecycle regardless of which option you chose.

3.1 Reconnaissance

- **Passive recon:** gathering information without touching the target directly — OSINT via search engines, WHOIS, DNS records, LinkedIn/company websites (for phishing pretext research), Shodan/Censys for internet-facing assets.
- **Active recon:** direct interaction that is detectable - ping sweeps, port scans (Nmap host discovery: -sn), banner grabbing.

For a phishing project, "reconnaissance" instead means researching the organization's naming conventions, email format, and current events to craft a believable pretext.

3.2 Scanning

- Port scanning: identify open TCP/UDP ports (Nmap -p-, -sU).
- Service/version detection: fingerprint what's actually listening (Nmap -sV).
- Vulnerability scanning: match services/versions against known-CVE databases (Nessus, OpenVAS, Nmap NSE vuln scripts).
- Enumeration: deeper interrogation of a discovered service — SMB shares, SNMP strings, web directories (gobuster/dirb).

See Figure/Exhibit A (sample Nmap output) and Exhibit B (sample web scan output) in the companion report for how this evidence is presented.

3.3 Exploitation (Controlled)

"Controlled" is the operative word for a capstone: exploitation exists to prove impact, not to cause damage.

- Always snapshot VMs before exploitation attempts so the environment is reversible.
- Prefer non-destructive proof-of-concept techniques: e.g., confirming code execution by writing a harmless file or popping a calculator, rather than deploying real ransomware behavior.
- Common technique categories: known-CVE exploits (e.g., EternalBlue/MS17-010), weak/default credential login, misconfiguration abuse (writable SMB share, exposed admin panel), and - for the phishing track - the "exploit" is simply a user submitting credentials to your controlled capture page.
- Privilege escalation: kernel exploits, misconfigured service permissions, credential reuse - document but avoid pivoting into anything outside the approved scope.

3.4 Documentation of Findings

Every finding should be evidence-backed and reproducible. A defensible finding record includes:

Field	Description
Finding ID	Unique tracking number (e.g., VULN-014)
Title	Short descriptive name
Affected Asset	Hostname/IP/URL
CVE / CWE reference	If applicable
CVSS Score & Severity	Numeric + Critical/High/Medium/Low
Evidence	Screenshot, log excerpt, tool output file reference

Field	Description
Reproduction Steps	Exact commands/steps to reproduce
Business Impact	What could an attacker actually achieve
Recommended Mitigation	Specific, actionable fix

3.5 Mitigation Strategies

Mitigations should be specific and prioritized, not generic "patch everything" statements. Group them by defense-in-depth layer:

- Preventive: patch management/WSUS, disabling legacy protocols (SMBv1), strong password policy/MFA, secure coding fixes.
- Detective: SIEM correlation rules, EDR deployment, centralized logging, honeypots/honeytokens.
- Corrective: incident response runbooks, backup/restore testing, segmentation to limit blast radius.
- Administrative/Human: security-awareness training tied directly to phishing-simulation results, least-privilege access reviews, vendor patch SLAs.

4. Incident Response Simulation

4.1 Governing Frameworks

- NIST SP 800-61 Rev. 2: "Computer Security Incident Handling Guide" - defines the four macro-phases: Preparation; Detection & Analysis; Containment, Eradication & Recovery; Post-Incident Activity.
- SANS PICERL model: Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned - a widely taught six-step variant of the same lifecycle (see Figure 3 in the companion report).
- MITRE ATT&CK: a knowledge base of real-world adversary tactics/techniques (e.g., T1110 Brute Force, T1566 Phishing, T1210 Exploitation of Remote Services) used to label what you detect, which makes your report immediately readable to any security professional.

4.2 Detecting the Attack in Logs

Detection is the process of correlating raw log events into a meaningful security alert.

Key log sources:

- Windows Security Event Log (Event IDs 4624/4625 logon success/failure, 4688 process creation).
- Sysmon (process creation with command line, network connections, file creation - much richer than default Windows logging).
- Firewall/IDS logs (Suricata/Snort alerts, blocked/allowed connections).
- Mail gateway logs (for the phishing track - who received/opened/clicked).

- SIEM correlation (Wazuh, Splunk, ELK/Elastic Security) - the layer that turns many small log lines into one prioritized alert, as shown in the sample SIEM alert exhibit in the companion report.

Indicators of Compromise (IOCs) to look for: unusual authentication patterns (many failures then a success), unexpected outbound connections, known-malicious hashes/IPs, execution of tools like PsExec/Mimikatz, or - for phishing - a spike of identical inbound emails followed by outbound connections from multiple hosts to the same unfamiliar domain.

4.3 Containment

- Short-term containment: isolate the affected host from the network (disable switch port / EDR network-isolation), block the malicious IP/domain at the firewall, disable the compromised account.
- Long-term containment: rebuild from known-good images while investigation continues, apply temporary compensating controls (e.g., force MFA, restrict lateral movement via segmentation) before full eradication.

4.4 Eradication

- Remove the root cause: uninstall/patch the vulnerable software, remove attacker-created accounts/scheduled tasks/persistence mechanisms.
- Reset all potentially exposed credentials, not just the obviously compromised one.
- Validate removal with a re-scan before declaring the system clean.

4.5 Recovery

- Restore the system to production/lab use, with enhanced monitoring for a defined observation period.
- Confirm business functionality and that no backdoors remain.

4.6 Writing the Post-Incident Report

A post-incident report is distinct from a vulnerability report — it tells the story of one specific event, in time order, with an emphasis on response effectiveness. Structure:

- ① Incident Summary - what happened, in one paragraph.
- ① Timeline - timestamped sequence from initial compromise/detection through recovery.
- ① Detection Method - what alert/log fired, and how long it took (Mean Time to Detect).
- ① Actions Taken - containment, eradication, recovery steps, and by whom.
- ① Root Cause Analysis - the underlying vulnerability or human factor exploited.
- ① Impact Assessment - systems/data affected, business impact (even if simulated).
- ① Lessons Learned / Recommendations - concrete process or control improvements, feeding back into Preparation for next time.

❖ Conclusion:

capstone project and incident response was successfully configured. This task is ready to support subsequent penetration-testing exercise.

❖ Deliverables:

Professional_capstone_report